基础类型 & 字节

类型	字节	范围/备注
bool	1	true/false
char	1	−128~127 **注意是有符号**
unsigned char	1	0~255
short	2	−32768~32767
int	4	-2^{31} $2^{31}-1$ 约 21 亿
long long	8	-2^{63} $2^{63}-1$
float	4	7 位有效数字
double	8	15 位有效数字
size_t	8	0 $2^{64}-1$，无符号

⚠ 脑残易错：表达式合法性（期中高频错题）

以下哪些声明/语句合法？为什么？

- `int &r;` → ✗ 引用必须初始化
- `const int c;` → ✗ const 必须初始化
- `static int s;` → ✓ 默认初始化为 0
- `Student s();` → ✗ 声明函数！非创建对象。用 `Student s;` 或 `Student s(args);`
- `class Foo { int x; }` → ✗ 类定义后缺分；
- `int a[5]; a[5] = 0;` → ✗ 越界 UB，合法索引 0–4
- `int *p; *p = 5;` → ✗ 野指针未指向任何对象
- `a[i] = i++;` → ✗ UB，赋值与自增未定义顺序
- `int *p = new int[5]; delete p;` → ✗ 应为 `delete[] p;`，不匹配是 UB
- `int x = (a > b) ? a : b;` → ✓ 三目运算符
- `int a = 1, b = (a++, a+1);` → ✓ 逗号表达式 b=2
- `if (a = 5)` → ✓（语法合法但逻辑通常错误，赋值返回 5→true）
- `int &r = 5;` → ✗ 不能绑定字面量，`const int& r = 5;` ✓
- `int* p = &a; int q = *p;` → ✓ p 指向 a，解引用得 a 的值
- `const int* p = &a; *p = 10;` → ✗ 指向 const 不能修改
- `int* const p = &a; p = &b;` → ✗ p 本身 const 不能重绑定
- `char s[] = "hello"; s[0] = 'H';` → ✓ s 是字符数组可修改
- `char* s = "hello"; s[0] = 'H';` → UB，字符串字面量是 const
- `int a = 5; int b = a++;` → ✓ b=5, a=6
- `int a = 5; int b = ++a;` → ✓ b=6, a=6
- `int a = 5/0;` → 编译不报错，运行时报错（除零）
- `Base* p = new Derived; p->f();` → 若 f 虚→多态 OK；若 f 非虚→调 Base::f()
- `Base b = *p; b.f();` → ✗ **对象切片**！丢失派生数据
- `delete[] new int[5];` → ✓ 配对正确
- `for (int i : {1,2,3}) cout<<i;` → ✓ C++11 initializer_list
- `int a[3]; a = {1,2,3};` → ✗ 数组不能整体赋值
- `int f(int a){return a;} double f(double a){return a;}` → ✓ 合法重载
- `int f(int a){return a;} double f(int a){return a;}` → ✗ 仅返回值不同
- `short s = 32768;` → overflow，short 范围-32768~32767

运算/隐式转换陷阱

- `5 / 2 = 2`（整数截断），`5.0 / 2 = 2.5`（隐式转 double）
- `-7 % 3 = -1`（C++11 向零截断），不同编译器可能不同
- `char c = 200;` → 溢出！char 范围-128~127，用 `unsigned char`
- `int a = 1000000 * 1000000;` → int 溢出！int 约 21 亿，超了
- 浮点用 eps 比较：`fabs(a-b) < 1e-9`；不能直接用 `==`
- `unsigned i; for(i=10; i>=0; i--)` → **死循环**（无符号永≥0）
- `bool b = -1;` →true（非零即真）
- `int a = 3.14;` →a=3（隐式截断，可能 warning）
- `sizeof(string)` 是固定大小（内部指针指向堆），不等于 strlen

输入输出

```
cin >> a >> b; cout << x << endl;
scanf("%d %lf %s %c", &n, &d, str, &c);  // 注意 &
printf("%.2lf %02d %8.3lf\n", d, n, d);
```

加速：`ios::sync_with_stdio(false); cin.tie(nullptr);`（之后勿混用 scanf/printf） argc/argv：`int main(int argc, char** argv)`，argc 参数个数，argv[0] 程序名 >> + getline 陷阱：>> 留 \n 在缓冲区，getline 读到空行。用 `cin.ignore(numeric_limits<streamsize>::max(), '\n');` 清除。 cin 读到 EOF 返回 false，可做 `while(cin >> x)`。

关键语法坑

宏括号：`#define SQR(x) ((x)*(x))`，不双括号 `SQR(1+2)→1+2*1+2=5≠9` 头文件保护：`#ifndef _HEAD_ / #define _HEAD_ / #endif` 或 `#pragma once` `#include <file>` 搜系统路径，`#include "file"` 先搜当前目录再搜系统路径 声明 vs 定义：`extern int a;` 声明（可多次），`int a=5;` 定义（只能一次）默认参数从右往左：`void f(int a, int b=0, int c=1);` 调用 `f(1)`，`f(1,2)` 函数重载：参数类型/个数/顺序不同，仅返回值不同不算重载！

指针 & 引用

```
int a = 10;
int *p = &a;        // 指针，存地址
int &ref = a;       // 引用，必须初始化，a的别名
ref = 20;           // a = 20
*p = 30;            // a = 30
```

引用 vs 指针：

	引用 &	指针*
可为空	否	是(nullptr)
可重绑定	否	是
必须初始化	是	否

const 引用可绑定临时对象：`const string& s`。指针运算：`p+1` 移动 `sizeof(*p)` 字节。`*(arr+i) = arr[i]`。 数组退化：数组传参退化为指针，丢失长度。函数内 `sizeof(a)` 是指针大小（8 字节），不是数组大小！

```
void f(int a[8]) { sizeof(a); } // sizeof(int*)=8, 不是8*sizeof(int)=32
```

野指针：delete 后未置 nullptr；返回局部变量地址（悬空指针）。 空指针：C++11 用 `nullptr`，替代 `NULL` 和 `0`。

内存管理

```
int *p = new int(5);        // 一个int初始化为5
int *arr = new int[100];    // 100个int
delete p; delete[] arr;     // 配对：new→delete, new[]→delete[]
```

必须配对！ 不匹配是 UB。delete 两次是 UB。malloc/free 不调构造/析构，C++优先用 new/delete。

位运算

```
& | ^ ~ << >>
int bit = (n >> k) & 1; n |= (1 << k); n &= ~(1 << k); n ^= n;
```

字符串

```
#include <cstring>
strlen(s); strcpy(d,s); strcat(d,s); strcmp(s1,s2); // 0相等 <0 s1<s2 >0 s1>s2
#include <string>
string s = "hello"; s.length(); s.size(); s += " world";
s.substr(0,3); s.find("ll"); // 返回位置或 string::npos
s.c_str(); stoi("42"); stod("3.14"); to_string(42); getline(cin, s);
```

终止符：string 内部多存 \0，strlen 不计入。c_str() 指针在修改 string 后可能失效！

生存期 & 作用域

创建	销毁	示例
自动 进入作用域	离开作用域	局部变量
静态 程序启动	程序结束	全局/static 局部
动态 new	delete	int*p=new int;

作用域 ≠ 生存期：作用域=编译期名字可见范围，生存期=运行时对象存在时间。 static 局部变量：只初始化一次，跨调用保持值（作用域仍在函数内）。全局变量：在所有函数外定义，程序全程存在。滥用导致难以理解和测试。 RAII：构造获取资源，析构释放资源。vector 用 RAII 管理内存。

编译 & Make & 分离编译

```
g++ -c main.cpp -o main.o   # 编译→.o
g++ -o main main.o          # 链接
g++ -std=c++17 -Wall -o main main.cpp
```

```
all: main
main: main.o stu.o
	g++ -o main main.o stu.o
clean:
	rm -f main *.o
```

分离编译：.h 放声明，.cpp 放定义。`ClassName::func` 作用域解析。 头文件改变→引用该头文件的所有源文件全部需重编译。单个 .cpp 改→只重编译该 .cpp。

STL：常用容器

```
#include <vector>
vector<int> v = {1,2,3}; v.push_back(4); v.pop_back();
v.size(); v.empty(); v.clear(); v.front(); v.back();
v[i]; // 不检查越界UB    v.at(i); // 检查越界抛out_of_range
v.reserve(n); // 预分配

#include <map>
map<string,int> m; m["key"]=42; m.count("key"); m.find("key");

#include <unordered_map>  // 哈希表O(1)平均
unordered_map<int,int> um;

#include <set>
set<int> st; st.insert(3); st.count(3); st.erase(3);

#include <queue>
queue<int> q; q.push(1); q.front(); q.pop();
priority_queue<int> pq;   // 大顶堆
priority_queue<int,vector<int>,greater<>> pq2; // 小顶堆
pq.top(); pq.pop();

#include <stack>
stack<int> sk; sk.push(1); sk.top(); sk.pop();

#include <deque>
deque<int> dq; dq.push_front(1); dq.push_back(2);

#include <utility>
pair<int,int> p = {1,2}; p.first; p.second;
```

STL：算法

```
#include <algorithm>
sort(v.begin(), v.end());
sort(v.begin(), v.end(), greater<>()); // 降序
nth_element(v.begin(), v.begin()+k, v.end()); // 第k小
swap(a,b); min(a,b); max(a,b);
find(v.begin(), v.end(), val); // 返end() 若未找到
count(v.begin(), v.end(), val);
reverse(v.begin(), v.end());
unique(v.begin(), v.end()); // 去重(先sort)返新末尾
lower_bound(v.begin(), v.end(), val); // ≥val第一个
upper_bound(v.begin(), v.end(), val); // >val第一个
binary_search(v.begin(), v.end(), val); // bool
find_if(v.begin(), v.end(), pred); // 必须检查!=end()!
#include <numeric>
accumulate(v.begin(), v.end(), 0); // 求和
```

自定义比较：`bool cmp(const T&a, const T&b){return a.x<b.x;}` cmp 必须满足严格弱序：若 `cmp(a,b)==true` 则 `cmp(b,a)==false`。 lambda 表达式：`sort(v.begin(),v.end(),[](const auto&a,const auto&b){return a>b;});`

排序算法对比

	最好	平均	最坏	稳定	原地	适用
冒泡	O(n)	$O(n^2)$	$O(n^2)$	是	是	教学
选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	否	是	交换少
插入	O(n)	$O(n^2)$	$O(n^2)$	是	是	近乎有序
快排	O(nlogn)	O(nlogn)	$O(n^2)$	否	是	库内部
sort	O(nlogn)	O(nlogn)	O(nlogn)	是	是	**默认**
stable_sort	O(nlogn)	O(nlogn)	O(nlogn)	是	否	需保序

稳定性：相等元素排序后保持原顺序。冒泡/插入稳定，选择/快排/sort 不稳定。 冒泡提前终止：一趟无交换(swapped=false)则已有序（最优 O(n)）。 选择排序交换次数最多 n-1 次，比冒泡少。插入排序适合近乎有序数据。 std::sort 内部 introspective sort（快排→堆排→插入 sort），保证 O(n log n) 最坏。 Big-O 直觉：n=10 时 $O(n^2)$=100 次比 $O(n\log n)$ 33 次差不太多；n=1000 时差 15 倍；n=10000 时差 750 倍。

std::vector 要点

`size()`=已用元素数，`capacity()`=已分配空间。`push_back` 触发重分配→之前所有指针/引用/迭代器失效！ `reserve(n)` 预分配避免多次重分配。 `[]` 不检查越界(UB)，`at()` 检查越界抛 `out_of_range`。`shrink_to_fit()` 请求释放多余容量。 2D 模拟：

```
vector<vector<char>> board(BOARD_SIZE, vector<char>(BOARD_SIZE, '.'));
board[r][c]; // 行访问
```

每行独立内存块（非一整块 R×C 连续内存），对棋盘清晰但对缓存不友好（数值计算用扁平 `vector<T>(R*C)+index=r*C+c`）。

迭代器与区间

算法操作 `[begin, end)` 半开区间：end 是最后一个元素的下一个位置。

```
for(vector<int>::iterator it=v.begin(); it!=v.end(); ++it) cout<<*it;
for(auto it=v.begin(); it!=v.end(); ++it) cout<<*it;  // auto 简化
for(const auto& x : v) cout<<x;  // range-for 最简洁
```

迭代器失效：push_back/insert/erase 可能导致迭代器失效，之后不可再用。

防御性编程

访问检查：

	危险	安全
越界	v[i] UB	v.at(i) 抛异常
搜索	*it 不检查	if(it!=v.end())
弹栈	pop_back() 空	if(!v.empty())
文件	不检查打开	if(!stream)

用户 replay 文件用 `at()` + try/catch。坐标/索引用 `isValidSquare` 验证。 delete 后置 `nullptr` 防野指针。

文件 I/O

```
#include <fstream>
ofstream out("f.txt");              // 写，默认截断
if(!out){/*错误*/} out<<42<<'\n';
ifstream in("f.txt");               // 读
int n; while(in>>n){/*...*/}
getline(in, line);                  // 读整行含空格
// 二进制
ofstream bin("f.bin", ios::binary);
bin.write(reinterpret_cast<const char*>(&val), sizeof(val));
ifstream f("f.bin", ios::ate|ios::binary);
long long sz = f.tellg();           // 文件字节大小
```

文本 vs 二进制：文本 42→2 字节字符，二进制 int=4 字节原样。序列化：顺序必须一致！不能 `write(vec.data(), sizeof(vec))`（vector 有内部指针）。 版本标记：`out<<1<<'\n';` 第一行写格式版本号。 POD 结构（纯 int/float 无指针）：可 `write(&p, sizeof(p))`。 打开模式：ios::app 追加，ios::ate 定位末尾，ios::trunc 截断，用 | 组合。

MVC 架构

Model：数据+规则，不依赖图形库。View：渲染，只读 const&。Controller：处理输入调 Model。View 不改 Model，Model 不碰图形，Controller 不画图。

结构体 & 枚举 & 命名空间 & 异常

```
struct Point{double x,y;}; // 默认public
union Data{int i;double d;}; // 共享内存
enum Color{RED,GREEN,BLUE};
enum class Dir{UP,DOWN,LEFT,RIGHT}; // 强类型，必须 Dir::UP
namespace NS{int x;void f();}
using namespace NS; // 慎用
try{throw runtime_error("err");}
catch(const out_of_range&e){e.what();}
catch(const exception&e){}
catch(...){} // 不确定异常时的函数
```

杂项

`this`：return *this; 链式调用。`sizeof`：编译期，类含对齐补齐。`mutable`：const 函数中可修改。 智能指针：`unique_ptr<T> p(new T);` 独占；`shared_ptr<T> p=make_shared<T>(args);` 引用计数。 `arrow->` 等价 `(*ptr).`。`explicit` 禁隐式构造。`constexpr` 编译期常量。

Lambda 表达式

```
// [capture](params)->ret{body}
auto f = [](int x){return x*x;};          // 无捕获
auto g = [&](int x){return x+n;};         // 引用捕获全部
auto h = [=](int x){return x+n;};         // 值捕获全部
auto k = [n,&m](int x){return x+n+m;};    // 混合捕获
auto l = [=]()mutable{n++;};              // mutable允许改值捕获变量
```

[=] 值捕获 → lambda 内变量是拷贝，不可修改（除非 mutable）。 [&] 引用捕获 → 注意悬垂引用（lambda 活得比变量久→UB）。 泛型 lambda（C++14）：`auto f=[](auto a,auto b){return a+b;};`

constexpr & const

- `const`：运行期不可修改，值可在运行期确定
- `constexpr`：编译期常量，必须编译期可求值

```
const int n = foo();        // OK，foo()运行期调用
constexpr int y = foo();    // 仅当foo()是constexpr时才OK
constexpr int arr[10];      // 编译期数组大小必须常量
```

constexpr 函数可在编译期执行（如参数是编译期常量）。 C++14 起 constexpr 函数可含循环和局部变量。`if constexpr`（C++17）：编译期条件分支。

RAII（资源获取即初始化）

构造函数获取资源，析构函数释放资源——异常安全的基础。

```
class FileGuard{ FILE* f;
public: FileGuard(const char* p):f(fopen(p,"r")){}
    ~FileGuard(){if(f)fclose(f);} // 自动释放
};
void process(){
    FileGuard fg("data.txt"); // 异常或return都安全
    // ... 不用手动fclose
}
```

标准库：`lock_guard`, `unique_lock`, `ifstream` 等全是 RAII。

迭代器失效 & erase

vector: `push_back` 扩容→全部失效；`insert/erase`→插入点之后失效。 list/map/set: `insert` 不失效；`erase` 仅失效被删元素迭代器。 正确写法：`it = v.erase(it);`（返回下一个有效迭代器）。

OOP：类 & 构造析构

```
class Obj {
public: int x; string name;
    Obj():x(0),name(""){}              // 默认构造
    Obj(int a,string n):x(a),name(n){}  // 带参
    Obj(const Obj& o):x(o.x),name(o.name){} // 拷贝
    ~Obj(){}                           // 析构，无返值无参
};  // 类定义末尾必须分号！
```

构造调用：

代码	调用
Obj a;	默认构造
Obj a(1,"x");	带参构造
Obj b=a; / Obj b(a);	拷贝构造
func(a); 值传递	拷贝构造
return a;	拷贝/移动
a=b;	拷贝赋值 operator=

初始化列表必须用于：const 成员、引用成员、无默认构造的成员对象、基类初始化。 **初始化顺序：声明顺序，非列表顺序**（脑中经典错）：

```
class Bad{ int b; int a; Bad():a(1),b(a){} }; // b先初始化！此时a未初始化→UB
```

`explicit` 禁隐式转换。`=default` 保留默认，`=delete` 删除。对象数组需默认构造。

⚠ 脑残易错：构造 & 析构

- `id=id;` → 参数遮蔽成员！用 `this->id=id` 或初始化列表
- 构造中调虚函数→**不会多态**（派生类未构造完，vptr 指向当前类的 vtable）
- 析构不能抛异常，无参数无返回值不可重载
- `Obj()` 创建临时对象非声明变量！`Obj p;` 才是声明
- 数组元素析构按逆序（最后一个先析构）
- 基类析构函数必须是 virtual 的！否则 `delete basePtr` 不调派生析构

static & const 成员

```
class A{
    static int cnt;             // 类共享，类外定义
    const int id;               // 每对象一份不可改
    static const int M=50;      // 整型可类内初始化
};
int A::cnt=0;                   // 必须类外定义！最易忘
```

static 成员函数：无 this，只能访问 static 成员。const 成员函数：`void print() const;` 不修改成员。 mutable：const 函数中仍可修改。const 三位置：`const int*` 指常，`int* const` 常指，`const int* const` 双常。

封装 & 访问控制

	同类	派生类	外部
private	✓	✗	✗
protected	✓	✓	✗
public	✓	✓	✓

Getter/Setter 模式：私有数据+公有接口，setter 可验证。struct 默认 public，class 默认 private。

继承

```
class Derived:public Base{ // public继承="is-a"
public: Derived(int a):Base(a){} // 显式调基类构造
};
```

构造顺序：基类→成员对象(声明序)→派生类体。析构顺序：(LIFO)派生体→成员(反序)→基类。 三种继承访问变化：

	public	protected	private
public 继承	pub	pro	inacc
protected 继承	pro	pro	inacc
private 继承	pri	pri	inacc

多重继承：`class C:public A,public B{};` 菱形→虚继承 `class B:virtual public A{};` 最底层类直接初始化虚基类 `D():A(x),B(x),C(x){};` is-a(继承空心三角)vs has-a(组合实心菱形/聚合空心菱形)。UML：+pub -pri #pro。

虚函数 & 多态

```
class Base{public: virtual void f(){} virtual ~Base(){}}; // 基类析构virtual!
class Derived:public Base{public: void f() override{}};
Base*p=new Derived(); p->f(); // Derived::f()动态绑定
delete p; // 虚析构确保~Derived
```

纯虚：`virtual void f()=0;`→抽象类不可实例化。override：编译器检查。final：阻止重写/继承。 vtable：含虚函数类有虚表，对象存 vptr。静态绑定(编译期)→看指针类型。动态绑定(运行时)→看实际对象。 **只有指针/引用才能多态**！`Base b=d;`→对象切片（派生数据丢失，多态失效）。 dynamic_cast 需要虚函数，失败返 nullptr(指针)或抛异常(引用)。

拷贝控制（三法则/五法则）

```
class PalString{ char* text;
public:
    PalString(const char*s){text=new char[strlen(s)+1]; strcpy(text,s);}
    PalString(const PalString&o){ // 深拷贝构造
        text=new char[strlen(o.text)+1]; strcpy(text,o.text);}
    PalString& operator=(const PalString&o){ // 深拷贝赋值
        if(this!=&o){delete[]text; // 自赋值检查！
            text=new char[strlen(o.text)+1]; strcpy(text,o.text);}
        return *this;}
    ~PalString(){delete[]text;}
};
```

三法则：需析构/拷贝构造/拷贝赋值之一→三个都需要。自赋值检查 `this!=&o` 不可忘！

friend & 运算符重载

```
class A{friend void show(A&a); friend class B; private:int s;};
// 友元不可继承、不可传递、不对称
Vec operator+(const Vec&b)const; // 双目成员
Vec& operator++();    // 前置++x 返引用
Vec operator++(int); // 后置x++ 返拷贝
friend ostream& operator<<(ostream&,const Vec&); // <<必须全局/友元
```

模板 & 类型转换

```
template<typename T> T mymax(T a,T b){return a>b?a:b;}
template<typename T,int N> class Stack{T data[N];}; Stack<int,100> s;
template<> void swap<MyType>(MyType&a,MyType&b); // 特化
static_cast<int>(d); dynamic_cast<Derived*>(bp);
const_cast<int&>(cr); reinterpret_cast<int*>(p);
```

模板编译期实例化，定义放头文件。隐式推导：`mymax(3,5)`→T=int；显式：`mymax<double>(3,5.0)`。

完整的构造/析构顺序示例

```
class Trace{string label; public:
    Trace(string s):label(s){cout<<"Creating:"<<label<<endl;}
    ~Trace(){cout<<"Destroying:"<<label<<endl;}
};
void nested(){          // 输出:
    Trace a("outer"); // Creating: outer
    {Trace b("in1");  // Creating: in1
     Trace c("in2");} // Creating: in2
                      // Destroying: in2  ← LIFO!
    Trace d("end");   // Destroying: in1
}                     // Creating: end
                      // Destroying: end
                      // Destroying: outer
```

继承链多态完整示例

```
class Person{protected:int id;string name;public:
    Person(int i,string n):id(i),name(n){}
    virtual ~Person(){=default;
    virtual void print()const{cout<<"Person:"<<name<<endl;}
};
class Student:public Person{ double gpa;public:
    Student(int i,string n,double g):Person(i,n),gpa(g){}
    void print()const override{cout<<"Student:"<<name<<" gpa:"<<gpa<<endl;}
};
class Professor:public Person{ string dept;public:
    Professor(int i,string n,string d):Person(i,n),dept(d){}
    void print()const override{cout<<"Prof:"<<name<<" dept:"<<dept<<endl;}
};
int main(){
    vector<Person*> people = {
        new Person(1,"Alice"), new Student(2,"Bob",3.5),
        new Professor(3,"Dr.Smith","CS")
    };
    for(Person* p:people){p->print(); delete p;} // 动态绑定！
}
// 输出: Person:Alice / Student:Bob gpa:3.5 / Prof:Dr.Smith dept:CS
```

抽象类 & pure virtual

纯虚函数 `=0` 使类变抽象类，不可实例化。纯虚函数可有实现（`Base::f()`），子类仍需 override。纯虚析构必须提供实现（即使 `=0`），否则链接错误。

override & final

`override` 让编译器验证确实覆盖了基类虚函数（防手误签名不匹配）。 `final` 阻止虚函数被进一步覆盖，或阻止类继承：`class A final{}`。

排序算法完整实现

冒泡排序：

```
void bubbleSort(vector<int>& v) {
    size_t n=v.size(); bool swapped;
    for(size_t i=0;i<n-1;++i) {
        swapped=false;
        for(size_t j=0;j<n-i-1;++j)
            if(v[j]>v[j+1]){swap(v[j],v[j+1]); swapped=true;}
        if(!swapped) break; // 已有序
    }
}
```

插入排序（理解）：

```
void insertionSort(vector<int>& v) {
    for(size_t i=1;i<v.size();++i) {
        int key=v[i]; size_t j=i;
        while(j>0 && v[j-1]>key){v[j]=v[j-1]; --j;}
        v[j]=key;
    }
}
```

选择排序：

```
void selectionSort(vector<int>& v) {
    for(size_t i=0;i+1<v.size();++i) {
        size_t minIdx=i;
        for(size_t j=i+1;j<v.size();++j)
            if(v[j]<v[minIdx]) minIdx=j;
        if(minIdx!=i) swap(v[i],v[minIdx]);
    }
}
```

稳定二级排序 comparator：

```
bool byDest(const Move&a,const Move&b){
    if(a.toRow!=b.toRow) return a.toRow<b.toRow; // 主键
    return a.toCol<b.toCol; // 次键
}
stable_sort(v.begin(),v.end(),byDest);
```

迭代器安全 & find_if

```
// 查找后必须检查!
auto it=find_if(v.begin(),v.end(),pred);
if(it!=v.end()){ /* 安全使用 *it */ } // 不检查解引用end()→UB

// find_if 示例：查找落子在第4行的move
bool landsOnRow4(const Move&m){return m.toRow==4;}
auto it=find_if(log.begin(),log.end(),landsOnRow4);
if(it!=log.end()) cout<<"Found at "<<(it-log.begin());
else cout<<"Not found";
```

防御性编程完整模式

```
bool undoLast(vector<Move>& log) {
    if(log.empty()){cout<<"Nothing to undo\n"; return false;}
    log.pop_back(); return true;
}
void printLastMove(const vector<Move>& log) {
    if(log.empty()){cout<<"No moves\n"; return;}
    cout<<"Last move to row "<<log.back().toRow<<endl;
}
bool isValidSquare(int row,int col) {
    return row>=0 && row<BOARD_SIZE && col>=0 && col<BOARD_SIZE;
}
bool addMoveSafe(vector<Move>& log, const Move& m) {
    if(!isValidSquare(m.fromRow,m.fromCol)
       || !isValidSquare(m.toRow,m.toCol)){return false;}
    log.push_back(m); return true;
}
```

异常 vs if：if 用于正常状态(空 log)，异常用于意外输入(用户输错索引)。

文件 I/O 完整示例

```
// 版本化存档
struct SaveGame{ int level; vector<int> inventory; };
void save(const SaveGame& g, const char* path) {
    ofstream out(path); out<<1<<'\n'; // 版本号
    out<<g.level<<'\n';
    out<<g.inventory.size()<<'\n'; // 长度前缀
    for(int id:g.inventory) out<<id<<'\n';
}
bool load(SaveGame& g, const char* path) {
    ifstream in(path); if(!in) return false;
    int ver; in>>ver; if(ver!=1) return false;
    in>>g.level; size_t n; in>>n;
    g.inventory.resize(n); // 预分配
    for(size_t i=0;i<n;++i) in>>g.inventory[i];
    return static_cast<bool>(in);
}
```

序列化规则：永远别 `write(&vec,sizeof(vec))`；逐字段序列化；读写顺序一致。

二进制 POD struct 读写

```
struct PlayerPod{int id=0; float x=0.f; float y=0.f;}; // POD!
// 写入
ofstream out("p.bin",ios::binary);
out.write(reinterpret_cast<const char*>(&p), sizeof(p));
// 读取
ifstream in("p.bin",ios::binary);
in.read(reinterpret_cast<char*>(&q), sizeof(q));
```

仅基本类型 struct 可用（无 string/vector/虚函数/指针）。

MVC 完整示例结构

```
// Model (no raylib includes!)
class GameModel {
    int playerX_,playerY_,coinX_,coinY_,score_;
public:
    static constexpr int SPEED=5,WINDOW_W=800;
    int getPlayerX() const{return playerX_;}
    void movePlayer(int dx,int dy);
    void update(); // collision + scoring
    void saveToFile(const char*) const;
};
// View (const ref to Model)
class GameView {
public:
    void draw(const GameModel& m) const;
};
// Controller (handles input)
class GameController {
public:
    void handleInput(GameModel& m);
};
// main wires them together
```

菱形继承与虚基类完整示例

```
class Animal{public:string name; Animal(string n):name(n){}};
class Flyer:virtual public Animal{ // ← virtual!
public: int wingSpan; Flyer(string n,int w):Animal(n),wingSpan(w){}
};
class Swimmer:virtual public Animal{ // ← virtual!
public: int swimSpeed; Swimmer(string n,int s):Animal(n),swimSpeed(s){}
};
class Duck:public Flyer,public Swimmer{public:
    Duck(string n,int w,int s):Animal(n) // 直接初始化虚基类!
        ,Flyer(n,w),Swimmer(n,s){}       // Animal() 被忽略
};
Duck duck("Donald",50,10);
cout<<duck.name; // ✓ 无歧义！只有一份 Animal
```

⚠️ 头文件陷阱（期中易错）

```
// 双引号 "" vs 尖括号 <>
#include <iostream>    // 尖括号：搜系统标准路径
#include "myfile.h"    // 双引号：先搜当前目录，再搜系统路径

// 重复包含问题：
// a.h 包含 b.h
// main.cpp 包含 a.h 和 b.h
// → 无保护则 b.h 被包含两次，报"重定义"错误

// 解决：头文件保护（include guard）
#ifndef MYFILE_H
#define MYFILE_H
// ... 头文件内容 ...
#endif
// 或 #pragma once（非标准但广泛支持）
```

对象数组及隐患

```
Professor faculty[3]; // 调用3次默认构造！
// 若 Professor 没有默认构造函数 → 编译错误
// 必须提供默认构造或显式初始化：
Professor faculty[3]={
    Professor(1001,"Dr.A"),
    Professor(1002,"Dr.B"),
    Professor(1003,"Dr.C")};
// 数组析构：按逆序，faculty[2]先析构→faculty[0]最后
```

静态数组 vs 局部数组：全局数组静态存储期→程序全程；局部数组自动存储→离开作用域销毁。

指针数组与浅拷贝

```
Pawn* blackPawns[8]; Pawn* whitePawns[8];
for(int i=0;i<8;++i) blackPawns[i]=new Pawn(i);
// 错误！浅拷贝→两者指向同一对象
for(int i=0;i<8;++i) whitePawns[i]=blackPawns[i];
// 后果：whitePawns[0]->promote() 影响 blackPawns[0]
// 双循环 delete 时双重释放→UB/崩溃！

// 正确：深拷贝（各自 new）
for(int i=0;i<8;++i) whitePawns[i]=new Pawn(i);
```

深浅拷贝深入

编译器生成的默认拷贝是**逐成员拷贝**（member-wise copy）。对于基本类型（int/double等），复制值。对于指针成员，只复制地址→两个对象指向同一内存。浅拷贝后果：两个对象共享同一堆内存，一个析构释放后另一个析构再释放→double free UB。一个修改也会影响另一个。深拷贝：分配新内存并复制值。判断是否需要深拷贝：类是否有指向动态内存的指针成员？有则三法则。

更多脑残陷阱

- `short s=32768;` → 溢出（short 范围-32768 32767）
- `vector<string> v(5);` 创建 5 个空 string，不是 5 个"5"
- `sizeof(string)` 返回固定大小（内部指针+size/capacity），不等于字符串长度
- `cout<<s.c_str();` 之前不能修改 s（c_str() 可能失效）
- `int* p=malloc(sizeof(int));` → C++中不合法（需强转，不用 C 风格）
- `int f(){return 1;} double f(){return 1.0;}` → 不合法！仅返回值不同
- 类中 static 成员不占对象大小，static 成员函数不传 this
- 模板特化：`template<> void swap<MyType>(MyType&a,MyType&b);`
- `for(auto x:v)` vs `for(auto& x:v)`：前者拷贝，后者引用可修改
- `auto x = expr;` 推导去掉引用和顶层 const，`auto& x = expr;` 保留

构造函数隐式转换陷阱

```
class MyString{
public: MyString(const char*s){} // 非explicit
};
void print(const MyString& s){}
print("hello");   // ✓ 隐式转换 const char*→MyString
// 若加 explicit：explicit MyString(const char*s){}
print("hello");   // ✗ 编译错误！必须显式 print(MyString("hello"))
// 赋值同理：MyString s = "hello"; // explicit 时错误
```

标准库中大部分单参数构造都是 explicit 的。

异常处理最佳实践

```
void replayMove(const vector<Move>& log, int moveNumber) {
    try{ const Move& m=log.at(moveNumber);
         cout<<"Move to row "<<m.toRow<<endl;
    } catch(const out_of_range& e){
        cout<<"Invalid! Log has "<<log.size()<<" moves.\n";
    }
}
// 捕获顺序：最具体的 exception 先，通用后
// 最后 catch(...) 兜底
// 为什么析构不能抛异常？对象销毁中再抛→std::terminate()
```

运算符重载注意事项

- `operator=` 必须是成员函数（不能全局）
- `operator<<` / `operator>>` 通常全局/友元（左操作数是 ostream/istream）
- 前置 `++x` 返回引用（无拷贝），后置 `x++` 返回拷贝（有哑参 int）
- `operator[]` 通常提供 const 和非 const 两个版本
- `operator()` 返回新对象（const 成员函数），`operator+=` 返回 `*this`（非 const）
- C++ 不允许重载的操作符：`. ::  .* ?: sizeof typeid`

智能指针 (Smart Pointers)

```
#include <memory>
unique_ptr<int> p1 = make_unique<int>(42); // 独占所有权
auto p2 = move(p1); // p1变nullptr, 不可拷贝只可移动
shared_ptr<int> s1 = make_shared<int>(10); // 引用计数
shared_ptr<int> s2 = s1; // 引用计数=2, 最后释放
weak_ptr<int> w = s1; // 不增引用计数, 解循环引用
if(auto sp = w.lock()){ /* w.lock()返回临时shared_ptr */ }
```

- `make_unique`/`make_shared` 优先于 `new`（异常安全+一次分配）
- `shared_ptr` 控制块开销，`unique_ptr` 零开销
- 循环引用→`shared_ptr` 泄漏，用 `weak_ptr` 打破
- 绝不用 `auto_ptr`（C++11 已废弃）

移动语义 (Move Semantics)

```
class Buffer{ char* data; size_t sz;
public:
    Buffer(size_t n):data(new char[n]),sz(n){}
    Buffer(Buffer&& o) noexcept // 移动构造：窃取资源
        :data(o.data),sz(o.sz){ o.data=nullptr; o.sz=0; }
    Buffer& operator=(Buffer&& o) noexcept { // 移动赋值
        if(this!=&o){ delete[]data;
            data=o.data; sz=o.sz;
            o.data=nullptr; o.sz=0; }
        return *this; }
    ~Buffer(){delete[]data;}
};
Buffer b2 = move(b1); // b1资源被窃取
```

std::move 不移动任何东西！只是将左值转为右值引用。移动后原对象处于"有效但未指定"状态（可析构/赋值，不可读取）。 noexcept 移动构造很关键：vector 扩容时若移动非noexcept 则回退拷贝。

杂项补充

this 是 const 指针（不可重绑定），成员函数中隐式可用。 `sizeof(空类) = 1` 字节（C++保证每对象有唯一地址）。 空基类优化(EBCO)：空基类可不占额外空间。 `alignof(T)` 返回对齐要求。`alignas(n)` 指定对齐。 `decltype(expr)` 获取表达式类型（不求值）。 `static_assert(cond,msg)` 编译期断言。 lambda：`[capture](params)->ret{body}`。`[=]` 值捕获全部，`[&]` 引用捕获全部。

抽象类 & override/final 示例

```
class Shape{public:
    virtual double area()const=0; // 纯虚→抽象类
    virtual ~Shape()=default;     // 虚析构必须
};
class Circle:public Shape{double r; public:
    Circle(double r):r(r){}
    double area()const override{return 3.14159*r*r;}
};
// Shape s; ← 编译错误！抽象类不可实例化
```

纯虚析构即使 =0 也必须提供实现，否则链接错误。

```
class Base{virtual void f(int)const;};
class Derived:public Base{
    void f(int)const override; // ✓ 编译器验证签名
    void g() final; // 禁止后续子类再override
};
```

override 防手误，final 阻止覆盖/继承（class A final{}）。

dynamic_cast & RTTI

```
Base* bp = new Derived();
Derived* dp = dynamic_cast<Derived*>(bp); // 运行时检查
if(dp){ /* 安全 */ } // 失败返回nullptr
Derived& dr = dynamic_cast<Derived&>(*bp); // 失败抛bad_cast
if(typeid(*bp)==typeid(Derived)){/* RTTI判断 */}
```

需要虚函数表（≥1 个虚函数）。static_cast 无运行时检查但更快。

期中高频错题专项：表达式合法性判断

1. `int x=5,y=10; swap(x,y);` ✓
2. `int* p=&x; int*& rp=p;` ✓ rp 是 p 的引用
3. `const int& r=42;` ✓ const 引用可绑字面量
4. `int&& rr=42;` ✓ 右值引用绑字面量
5. `int& r=(a>b?a:b);` ✗ 三目返回右值，不能绑到普通引用
6. `const int& r=(a>b?a:b);` ✓ const 引用可绑
7. `vector<int> v(10,5);` ✓ 10 个元素都是 5
8. `vector<int> v{10,5};` ✓ 2 个元素 {10,5}
9. `int a=5; void* p=&a; int* q=static_cast<int*>(p);` ✓
10. `int x; cout<<x;` ✗ x 未初始化（局部变量值未定义）
11. `static int x; cout<<x;` ✓ 静态变量默认 0
12. `int a[5]={};` ✓ 全部初始化为 0
13. `string s="hello"; s[0]='H';` ✓ mutable string
14. `const string s="hello"; s[0]='H';` ✗ const 不可修改
15. `void f(int a[5]){cout<<sizeof(a);}` 输出指针大小(8)，非数组大小(20)
16. `char s[]="ab"; s[1]='c';` ✓ 字符数组可修改
17. `int a=1,b=2; int* p=&a; p=&b;` ✓ 指针可重绑定
18. `int a=1,b=2; int& r=a; r=b;` ✓ r 重绑定？NO！r=b 意思是 a=b！
19. `Obj a; Obj b=a; vs Obj a; Obj b; b=a;` 前者拷贝构造，后者拷贝赋值
20. `int f(); int (*pf)()=f;` ✓ 函数指针
21. `Base* p=new Derived; delete p;` 若无 virtual 析构→UB/泄漏
22. `int a=1; int b=(a++,++a,a+2); b=4` (逗号表达式返回最后一项)
23. `int a=1; cout<<(a++,a+2);` 输出 3，a=2
24. `if(int x=foo()){/*用 x*/}` ✓ C++17 if-with-init
25. `for(int i=0;i<n;i++) int x=i;` ✓ x 作用域只在循环体
26. `int a=1,b=2; int& r=a; r=b;` r 仍是 a 的引用，a 值变成 2
27. `class A{}; class B:private A{}; class C:public B{};` C 中无法访问 A 的 public 成员
28. `int* p=new int[5]; delete p;` 错！delete[]

C++ 版本特性速查

- C++11: auto, decltype, nullptr, 范围 for, lambda, constexpr, enum class, 智能指针, 初始化列表, =default/=delete, override, final, 右值引用 &&
- C++14: 泛型 lambda, 返回类型自动推导
- C++17: if constexpr, inline static, 结构化绑定, string_view, optional
- 课程使用：C++17 编译 `g++ -std=c++17`

模板进阶

```
// 非类型模板参数
template<typename T, size_t N>
class Array{ T data[N]; public: size_t size(){return N;} };
Array<int,100> a; // 编译期固定大小

// 默认模板参数
template<typename T=int, size_t N=10>
class Buffer{ T buf[N]; };
Buffer<> b; // Buffer<int,10>

// 模板特化
template<typename T> class MyClass{/*通用*/};
template<> class MyClass<int>{/*int特化*/};
// 偏特化：template<typename T> class MyClass<T*>{/*指针特化*/};
```

模板定义放头文件（编译期实例化需要完整定义）。每个实例化是独立类型。

static 成员详解

```
class Professor{
    static int totalProfessors; // 声明：所有对象共享
    string name;
public:
    Professor(string n):name(n){totalProfessors++;}
    ~Professor(){totalProfessors--;}
    static int getTotal(){return totalProfessors;} // 静态成员函数
    string getName()const{return name;} // 非静态需要this
};
int Professor::totalProfessors=0; // 必须在类外定义！(cpp文件中)
// 调用：Professor::getTotal()   // 无需对象
// 静态成员函数不能访问非静态成员(没有this)
```

this 指针详解

每个非静态成员函数隐含 `this` 指针指向调用对象。 类型：`T* const this`（指针本身是常量，不可重绑定）。

```
class A{int x; public:
    A& setX(int x){ this->x=x; return *this; } // 链式调用
};
A a; a.setX(1).setX(2); // 链式调用
```

const 成员函数中 this 是 `const T* const`。

虚函数表 (vtable) 机制

每个含虚函数的类有一张 vtable（虚函数表），对象存一个 vptr（虚表指针）指向该表。 调用 `p->f()` 时：通过 p 的 vptr 找到 vtable，查表得函数地址，调用。 代价：一次间接寻址（几乎可忽略）。因此多态有微小运行时开销。

const 指针辨析

声明	含义	口诀
`const int* p`	指向常量的指针（p 可变，**p 不可变**）	**const 在左→修饰对象**
`int* const p`	常量指针（p 不可变，**p 可变**）	**const 在右→修饰指针**
`const int* const p`	常量指针指向常量（都不可变）	两头 const
`int const* p`	等价于 `const int* p`	写法不同

const 成员函数：`int get() const`——承诺不修改成员。 const 对象只能调用 const 成员函数。非 const 对象两者皆可调。 mutable 成员：即使在 const 成员函数中也可修改。

enum class vs enum

```
// C风格enum：污染外部作用域
enum Color{RED,GREEN,BLUE}; // RED可直接用, 隐式转int
// C++11 enum class：强类型作用域
enum class Color{RED,GREEN,BLUE};
Color c = Color::RED;   // 必须限定
int y = static_cast<int>(c); // 显式转换
enum class Status:uint8_t{OK,ERR}; // 可指定底层类型
```

优势：类型安全、不隐式转 int、不污染作用域、可前向声明。

初始化方式速查

```
int a = 5;      // 拷贝初始化
int b(5);       // 直接初始化
int c{5};       // 列表初始化（推荐！禁止窄化 double→int）
int e{};        // 值初始化 → 0
struct S{int x; double y;};
S s{1, 3.14}; // 聚合初始化
vector<int> v{1,2,3}; // initializer_list 构造
vector<int> v2(10,5); // 10个5; 用()则是2个元素{10,5}
```

- `{}` 最安全：禁止窄化转换
- `()` vs `{}` 歧义是高频考点：`v(10,5)` vs `v{10,5}`
- 空 `{}` = 值初始化（内置类型→0）

nullptr & auto

```
int* p = nullptr; // C++11, 类型 nullptr_t
// nullptr 可隐式转任意指针类型, 不能转int
// NULL 是宏(0或0L), 重载歧义!
void f(int); void f(int*);
f(NULL);    // 可能调 f(int)！不确定
f(nullptr); // 一定调 f(int*) ✓

auto x = 42;          // int
auto& rx = x;         // int& (引用保留)
const auto& cr = x;   // const int&
auto y = rx;          // int (auto去掉引用和顶层const)
decltype(auto) z = rx; // int& (完整保留引用和cv)
```

Rule of Three / Five

三：析构+拷贝构造+拷贝赋值 → 三个都需要 五（C++11）：加 移动构造+移动赋值 零：能用标准库容器/智能指针就别自己管理资源

string 常用操作

```
string s="hello"; s+=" world"; // 拼接
s.substr(0,5);     // "hello" — 子串
s.find("wo");      // 返回索引或 string::npos
s.replace(0,5,"hi"); // 替换
s.compare("abc");  // 比较, 返回0表示相等
stoi("42"); stod("3.14"); to_string(42); // 转换
```

`c_str()` 返回 `const char*`，修改 string 后可能失效。 `string_view`（C++17）：不拥有内存的只读视图，传参首选。

算法复杂度速查

算法	时间	空间	备注
`sort`	$O(n \log n)$	$O(\log n)$	内省排序
`stable_sort`	$O(n \log n)$	$O(n)$	稳定
`binary_search`	$O(\log n)$	$O(1)$	需有序
`find` / `count`	$O(n)$	$O(1)$	线性
`lower_bound`	$O(\log n)$	$O(1)$	需有序，返迭代器

容器：`vector` 随机访问 $O(1)$，尾部增删 $O(1)$ 均摊；`list` 任意位置增删 $O(1)$；`set/map` 查找 $O(\log n)$（红黑树）；`unordered_set/map` 平均 $O(1)$（哈希）。

范围 for & 结构化绑定

```
for(auto& x:v)       // 引用→可修改元素
for(const auto& x:v) // 常引用→只读, 不拷贝
for(auto x:v)        // 拷贝→修改不影响原容器

// 结构化绑定 C++17
auto [x,y,z] = make_tuple(1,2.0,"hi"); // x=1, y=2.0, z="hi"
for(auto& [k,v] : map){ /* k=key, v=value */ }
pair<int,string> p{42,"ans"}; auto& [id,name] = p;
```

容器选择：随机访问→`vector`；频繁头尾增删→`deque`；任意位置增删→`list`；有序 key→value→`map`；快速查找→`unordered_map`。

总结：考试最常错的

易错点	正解
`Student s();`	是函数声明！应 `Student s;`
类定义缺 `;`	`class Foo{};` 分号必加
初始化顺序	声明序非列表序
static 成员定义	必须在类外 `int A::x=0;`
Base 析构非 virtual	多态 delete 泄漏
对象切片	用指针/引用传递
`new[]`配 `delete`	必须 `delete[]`
`v[i]` 越界	用 `v.at(i)` 安全
find_if 后不检查	必须 `!= end()`
浮点 `==` 比较	用 eps 比较
数组退化	传参丢失长度
`x/0` 编译通过	运行时错误
无符号死循环	`i>=0` 永真
默认参右往左	`void f(int a=0,int b)` 错
仅返回值重载	不合法